

LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 3
Student Name: A.Bhanu Prasad
Roll No.: A24126552068
Date: 30-08-2026

1. TITLE

JavaScript ES6 Array Methods and Object-Oriented Programming – Student Management System

2. AIM

To develop JavaScript programs for array manipulation (calculating total and average marks, adding elements) and implement an Object-Oriented Student Management System using an ES6 class with methods for adding, displaying, calculating average marks, and removing students.

3. OBJECTIVE

The objectives of this experiment are:

- To write modular JavaScript functions for array data processing.
- To implement total and average calculation algorithms using loops.
- To manipulate arrays dynamically using the `push()` and `pop()` methods.
- To define an ES6 class with a constructor and instance methods.
- To manage collections of student object literals inside a class instance.

4. THEORY

JavaScript provides robust array operations and ES6 Classes for Object-Oriented Programming. Functions encapsulate reusable logic, such as iterating over marks to compute totals and averages. Classes encapsulate state (the `this.students` array) and behaviors (`addStudent`, `displayStudents`, `averageMarks`, and `removeStudent`), enabling clean and modular software architecture.

Application Flow: Initialize Array/Class → Execute Processing Functions → Manage Student Objects → Print Output to Console

5. ALGORITHM / PROCEDURE

1. Create `functions.js` and define an array of student marks `[85, 90, 78, 88, 95]`.
2. Implement `displayMarks()`, `calculateTotal()`, `calculateAverage()`, and `addMark()`.
3. Calculate and log the initial total (436) and average (87.2).
4. Add a new mark (92) and log the updated total (528).

5. Create `object.js` and define an ES6 class `Student` with a constructor initializing an empty `students` array.
6. Implement `addStudent(name, marks)`, `displayStudents()`, `averageMarks()`, and `removeStudent()`.
7. Instantiate `Student`, add four students (Bhanu, Rahul, Sree, Karthik), display the list, calculate the average (87.5), remove the last student, and calculate the new average (85).
8. Execute the scripts using Node.js and verify the output.

6. PROGRAM

functions.js

```
let marks = [85, 90, 78, 88, 95];

function displayMarks(marks) {
    console.log("Student Marks:", marks);
}

function calculateTotal(marks) {
    let total = 0;
    for (let mark of marks) {
        total += mark;
    }
    return total;
}

function calculateAverage(marks) {
    return calculateTotal(marks) / marks.length;
}

function addMark(marks, newMark) {
    marks.push(newMark);
}

displayMarks(marks);
console.log("Total Marks:", calculateTotal(marks));
console.log("Average Marks:", calculateAverage(marks));
addMark(marks, 92);
console.log("After Adding New Mark:");
displayMarks(marks);
console.log("New Total:", calculateTotal(marks));
```

object.js

```
console.log("==== Student Management System =====");

class Student {
    constructor() {
        this.students = [];
    }

    addStudent(name, marks) {
        this.students.push({
            name: name,
            marks: marks
        });
        console.log(name + " added successfully.");
    }

    displayStudents() {
        console.log("\nStudents:");
        for (let i = 0; i < this.students.length; i++) {
            console.log(
                (i + 1) + ". " +
                this.students[i].name +
                " - Marks: " +
                this.students[i].marks
            );
        }
    }

    averageMarks() {
        let total = 0;
        for (let i = 0; i < this.students.length; i++) {
            total = total + this.students[i].marks;
        }
    }
}
```

```
    }  
    return total / this.students.length;  
  }  
  
  removeStudent() {  
    let student = this.students.pop();  
    if (student) {  
      console.log("\nRemoved Student: " + student.name);  
    }  
  }  
}  
  
let school = new Student();  
school.addStudent("Bhanu", 90);  
school.addStudent("Rahul", 80);  
school.addStudent("Sree", 85);  
school.addStudent("Karthik", 95);  
school.displayStudents();  
console.log("\nAverage Marks: " + school.averageMarks());  
school.removeStudent();  
console.log("\nAfter Removing Student:");  
school.displayStudents();  
console.log("\nNew Average Marks: " + school.averageMarks());
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>calculateTotal(marks)</code>	Uses a <code>for...of</code> loop to iterate through the array and calculate the cumulative sum of marks.
<code>calculateAverage(marks)</code>	Calls <code>calculateTotal()</code> and divides the total by the number of marks to obtain the average.
<code>marks.push(newMark)</code>	Appends a new mark to the end of the marks array.
<code>class Student {}</code>	Defines an ES6 class that encapsulates student data and related operations.
<code>constructor()</code>	Initializes the internal <code>students</code> array when a <code>Student</code> object is created.
<code>addStudent(name, marks)</code>	Creates a student object containing name and marks, then stores it using <code>push()</code> .
<code>displayStudents()</code>	Iterates through the <code>students</code> array and displays each student's name and marks.
<code>averageMarks()</code>	Adds all student marks using a loop and divides the total by the number of students.
<code>this.students.pop()</code>	Removes and returns the last student from the <code>students</code> collection.

8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Call <code>displayMarks(marks)</code>	Displays initial array [85, 90, 78, 88, 95]	Initial marks displayed	Pass
TC-02	Call <code>calculateTotal</code> and <code>calculateAverage</code>	Total: 436, Average: 87.2	Total 436, Average 87.2	Pass
TC-03	Call <code>addMark(marks, 92)</code>	Mark appended; new total: 528	92 appended; total 528	Pass
TC-04	Add 4 students to <code>Student</code> class	Students added and student list displayed	Four students displayed with marks	Pass
TC-05	Calculate average and remove last student	Initial average 87.5; after removal new average 85	Karthik removed; new average 85	Pass

9. OUTPUT

Output 1: JavaScript Array Functions Output (Total & Average Marks)

```
PS C:\\Users\\bhanu\\OneDrive\\Desktop\\fullstack> node functions.js
Student Marks: [ 85, 90, 78, 88, 95 ]
Total Marks: 436
Average Marks: 87.2
After Adding New Mark:
Student Marks: [ 85, 90, 78, 88, 95, 92 ]
New Total: 528
```

Output 2: ES6 Student Management Class Output (Student List & Average Marks)

```
PS C:\\Users\\bhanu\\OneDrive\\Desktop\\fullstack> node object.js
===== Student Management System =====
Bhanu added successfully.
Rahul added successfully.
Sree added successfully.
Karthik added successfully.

Students:
1. Bhanu - Marks: 90
2. Rahul - Marks: 80
3. Sree - Marks: 85
4. Karthik - Marks: 95

Average Marks: 87.5

Removed Student: Karthik

After Removing Student:

Students:
1. Bhanu - Marks: 90
2. Rahul - Marks: 80
3. Sree - Marks: 85

New Average Marks: 85
```

10. RESULT

Thus, JavaScript array manipulation functions and the ES6 Object-Oriented Student Management System were successfully developed, executed, and verified with accurate outputs.

— End of Experiment —